Front End Principles & Architecture

9 min read · Jan 12, 2023

 Jimmy Emery Follow

Introduction

To tell you a little bit about my history and how I got here, I first started my career as a High School Math Teacher and Lacrosse Coach and also ran multiple lacrosse camps for kids. I eventually made my way into the private sector and became a Senior Business Analyst for companies like Groupon and Avant, and while at Avant started learning Python. I soon fell in love with programming, and quickly transitioned to Avant's Machine Learning infrastructure team. I spent most of my nights after work teaching myself React because the idea of making the applications that people interact with, and to make them look really beautiful, well, seemed really cool.

I say all this because I think you learn something trying to teach 17 year olds calculus and 4 year olds how to catch and throw with a lacrosse stick (very difficult, beyond the fact that you need to get them to stop eating the grass … true story). I think you learn a lot from working with different sales teams and executive teams across a large e-commerce company, or from working with the finance and compliance department of a large FinTech.

I think you learn how to communicate and collaborate. I think you learn that not everybody understands your perspective or has your same knowledge or speaks your same lingo. I think you learn that it's ok to disagree, and it's also ok if your idea is wrong.

I guess that's what I've brought with me to writing front end applications. Is what I am doing clear? How do I communicate to designers what is required to implement their designs? How do I make sure that my code and all of my intentions of what I am trying to build is clear for the next developer?

Below, is, in a nutshell, what I think are some good principles to building and scaling a front end application. I tried my best to keep it relatively short and sweet. I could probably write paragraphs about almost each item on the lists below, but I didn't think that would be too digestible.

Documentation

- **Style Guide**
  - Naming conventions
  - Component composition
  - State composition
  - When to drill and when to use higher state

- CSS
- Data fetching & general component hydration

- **Recipes** (outlined structures of common code)
- Components
- Tests
- State

- **Other important documentation**
- Getting started guide
- List of current technical debt and their priorities
- Detailed guide of the architecture of the application
- Testing philosophy
- Infrastructure
- Detailed summary of NPM commands

Architecture

- **Philosophy**
- *Scalability*: support the application's continuous growth both in terms of volume and complexity.
- *Team growth*: if and when a development team grows, to enable multiple developers with various expertise levels to collaborate and develop on the application simultaneously. This should allow development to occur in the following ways: 1. Allow developers to easily work independently on the application without having to worry about other developers interfering with their code. This allows developers to not have to worry about learning or worrying about other areas of the application in which they do not own (multiple developers simultaneously building different components without having to worry about how other teammates are building those components and how said components work). 2. Enable teammates to collaborate easily and understand each other's work quickly given a development structure / architecture.
- *DRY-ability*: reduce redundancies in the codebase as well as make it easy to reduce future redundancies.
- *Learn-ability*: structure the application in such a way that it is easily understandable and easy for new developers to join and quickly begin developing. This will reduce the team's bus factor, making sure that one of our

high points of failure is not having to lose a developer with highly developed domain knowledge.

- **Principles**
  - Agnostic & independent
  - Re-usable
  - Single concern
  - Modular

- **Structure**
  - Feature specific code should be separated from common component code and common tooling code.
  - Avoid putting everything into a single `src/` folder with a generic `components/` folder and a generic `pages/` folder, etc. Be more structured, and try to break out the actual feature code by individual domain.
  - Consider atomic design where necessary.
  - Create common folder structures and common naming conventions to handle state, types, utilities, tests, components, CSS, and more. You will repeatedly run into these same patterns, so the more these patterns remain consistent, the more readable the code.

- **Repositories & Packages**
  - To scale the application, consider the front end code base either having multiple repos or being a monorepo that supports multiple feature-focused applications as well as multiple common packages. This will allow you to independently release and deploy each feature application and package, increasing speed and efficiency for production deployment and patch fixes.

- **Hydration Management**
  - The way that pages and components load and fetch data should be consistent across the application. This implementation often involves collaborating with design on how they want to load pages, but making sure data is fetched and stored in a consistent manner to components has large architectural implications. React's v18 release with concurrent rendering has provided a significant api enhancement that should consider getting used.

- **State**
  - Follow the principles above; make sure it is independent and of single concern.
  - Try and keep the state colocated in the component tree as low as possible.

Avoid a large global state with information that is unnecessary to other parts of the application and thus can cause unnecessary re-rendering.

- **Business Logic**
  - Keep the front end dumb and agnostic of the business logic and keep the business logic on the back end in preferably a dedicated layer. Components and features intertwined with business logic makes them difficult to refactor. Any time business logic needs to be changed, so does the front end presentational layer. If you suddenly have to deploy these features to new application forms such as mobile with different business logic, you incur a lot more debt.

Infrastructure

- Typechecking
- Linting (Prettier, ESLint, Stylelint); make sure to add a git hook to run linting on all commits.
- Automated test runs on all PRs
- Something like SonarCloud for code coverage, code quality, etc
- Automated performance evaluation like Calibre
- Storybook deployment pipeline (see more details in the Collaboration section)
- Github's CI seems pretty simple and straightforward to execute all of these, no longer much of a need for something like Circle CI.

APIs

- **State**
  - I'm a fan of Context because of it's flexibility and minimal overhead, but that also means it's flexibility allows for poor state architecture if not managed and designed well.
  - Redux is still commonly used for good reasons, although I think it's boilerplate is a little too thick and doesn't provide the flexibility of where individual states lie along the application tree.
  - There are of course a lot of alternatives like Recoil and Zustand. I find Legend-State pretty inspiring.
- **Component Libraries**
  - Some I'm a fan of: Mantine, Tailwind, Elastic UI

- **Storybook** (see more details in the collaboration section)
- I'm a big fan of **GraphQL**, and specifically what Apollo is building.
- I think it's obvious to mention **Next JS**. Next JS reminds me of TypeScript 4 years ago where everyone urgently felt the need to learn it. It feels like it will be here for a while. I still think you need to evaluate if and when you are going to use CSR vs SSR.
- I think **Expo** deserves a shout out, as it seems it is achieving a lot of the things for React Native that Next JS is achieving for React.
  - If you are going to build a React Native application in addition to a Web application, I do think there should be some consideration about if your entire front end application should be built in React Native.
- **Charting**
  - I've spent a lot of my time evaluating and implementing charting libraries. I'm a huge fan of visx, but it definitely takes a lot more time and resources to get the api off the ground as supposed to some other, more out-of-the box solutions.
  - Some other charting libraries I'm a fan of: Recharts, Victory
- **Evaluating APIs**
  - When implementing new APIs and libraries, especially libraries such as component libraries, I think it is critical to perform thorough research and evaluation on all of the options to make sure you don't end up with the wrong library 1 year down the road. Some important items to evaluate:
  - What are the features that you will need? If you will need a charting library with a scatter plot, or a component library with it's own paginated tables, it's important you don't pick a library that lacks these features.
  - Github downloads
  - Github stars
  - When was the latest release and how often do they release
  - License
  - Community support: how quickly do they address issues
  - How good is their documentation
  - Ease of integration
  - Additional concerns specific to the library and how it plans on getting used, such as performance, responsiveness, accessibility, customizability, etc.

Collaboration

- **Agile**
  - It's better, even in a very early stage startup, to implement an agile methodology. Especially for the front end where a lot of collaboration is required with design, product, back end, project management, and engineering management.
  - I've seen the mistake happen too many times where early stage teams don't track any work or any progress. This usually results in lots of ideas getting lost and multiple meetings discussing the same ideas over and over again because we ultimately forgot what we said two months ago. When building an application from the ground up and so many ideas and so much to consider, it's much better to be organized and put them down on paper so you can come back to them later.
  - Following sprints, even in an early stage, at least lets everyone know where the focus is and what progress is being made.
  - Some tools that really appeal to me: ClickUp, Monday, Asana

- **Storybook**
  - I think storybook is critical to collaborating with product, design, and other front end developers. It provides a clear api documentation to the common components and tools you are writing, and provides product and design real-world examples of the common features you are building.
  - I'm a fan of storybook development. What I mean by this is that I think more development, beyond just simple components, can be done inside of storybook. If our goal is to really make sure that more robust features are truly agnostic of business logic and are independent of other features that they are surrounded by, then we should be able to develop these features inside of storybook with mock data. This will allow product and design to test and evaluate these features more thoroughly in an independent manner, and also will encourage developers to create these features in an agnostic and independent manner.
  - Storybook should obviously be deployed to some static, possibly secured server, every time the front end code base is updated, but I think we should have additional storybooks for any and every relevant PR. If a developer makes changes to a button component and has a pending PR, than we can create a CD pipeline that deploys a custom storybook with those specific changes before the PR gets merged. This will allow product and design to easily review these changes, provide feedback, and request changes to the code before anything

gets merged. Product and design feedback usually happens after code gets merged and thus results in more PRs, more code reviews, and overall more time. This workflow should hopefully increase collaboration among teams and make sure that what we ship the first time is what everyone wants, thus increasing efficiency.

- **Transparency**
  - Front end can often be a black box that not enough people know what the daily inner workings are. With so many teams collaborating with front end (product, design, back end, project management, engineering management), the more understanding they have about where we are headed, the better. I think, at minimum, a monthly email to the relevant stakeholders provides really good value, and should include the following:
  - *What is the long term front end road map (next 3–6 months)*
  - *What did we accomplish last month*
  - *What did we say that we were going to accomplish last month and didn't, and why*
  - *What do we plan on accomplishing this month*
  - *What important debt is holding us back*

- **Technical Design Documents**
  - What apis are needed, and how should those apis be formatted
  - Are there any changes to currently built components or tools
  - Are there any new components or tools that are needed
  - Are there any unique UI complexities to consider

- **Working with:**
  - *Design:* make sure to communicate if, how, and why, any designs will be more complicated to implement that what they may first appear to be. I believe our role as front end developers is to encourage and empower designers to design the best product they can; it's up to us to determine how to build it.
  - *Project managers:* communicate clearly what you think your best timeline estimates are and how confident you are in those estimates.
  - *Engineering managers:* don't be afraid to speak up about any debt that you think will hurt the company long term if not addressed in a timely manner.
  - *Product:* make sure you have a clear understanding of what they want and why they want it.
  - *Back end:* discussing and collaborating ahead of time regarding how you think the apis should be structured and formatted will save every one a lot of time.

- **Pull Requests**
  - I think it is useful to have pull request templates that follow some common structure so developers know what they are reviewing and why.

Front End Development | Engineering | React | Software Development | Architecture

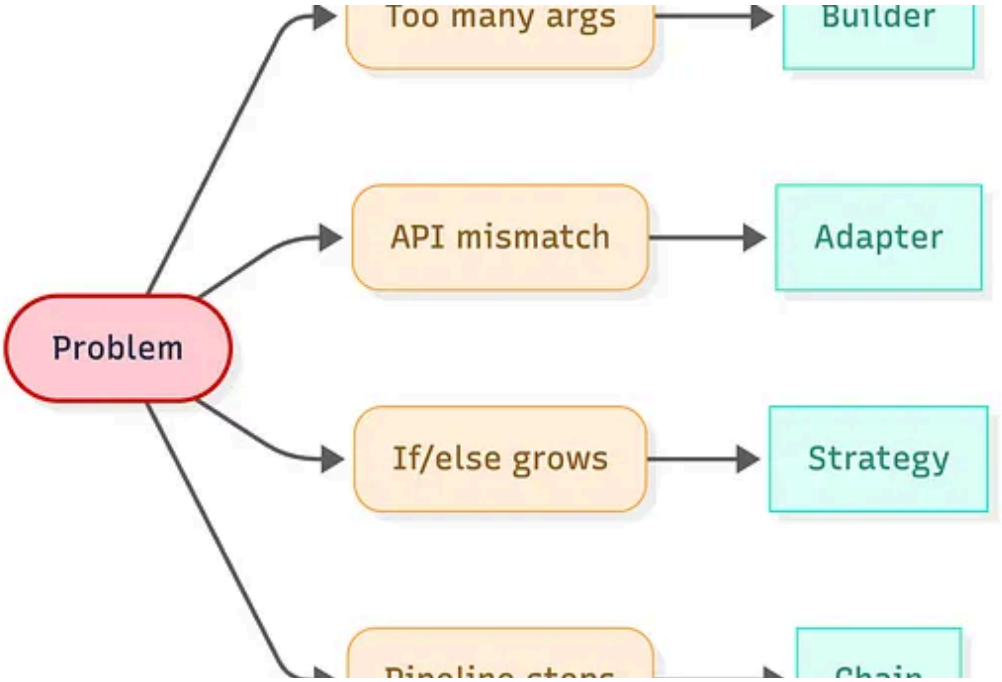
Follow

Written by Jimmy Emery

3 followers · 1 following

https://www.linkedin.com/in/jimmydemery/

Recommended from Medium

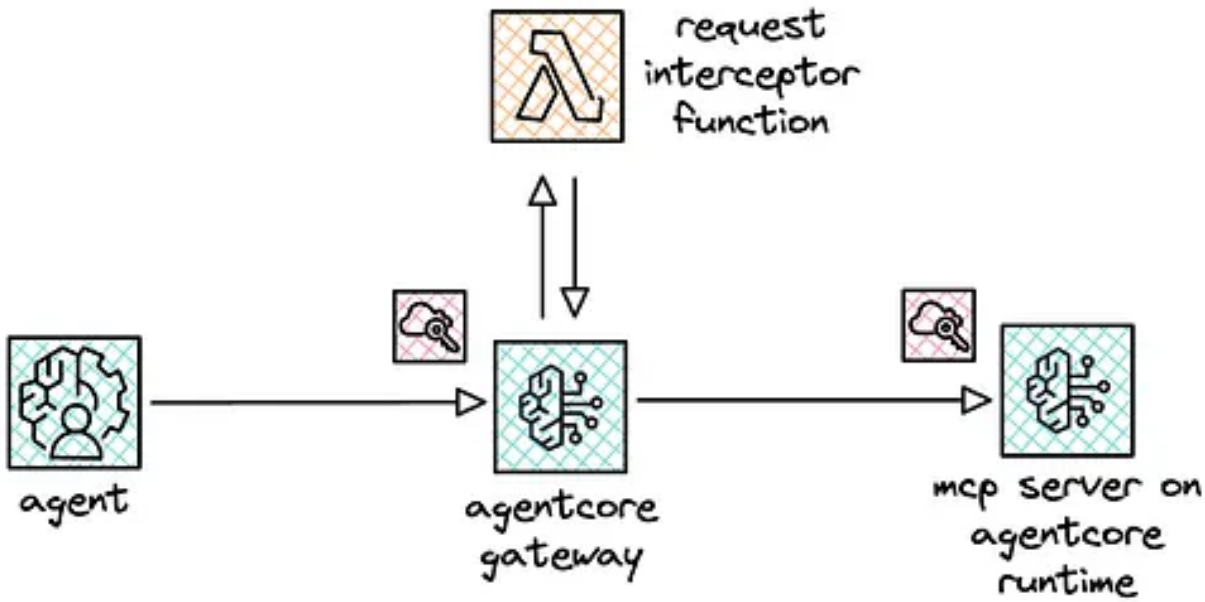


In Women in Technology by Alina Kovtun✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-engineering in any OO language.

Jan 29 6.5K 59

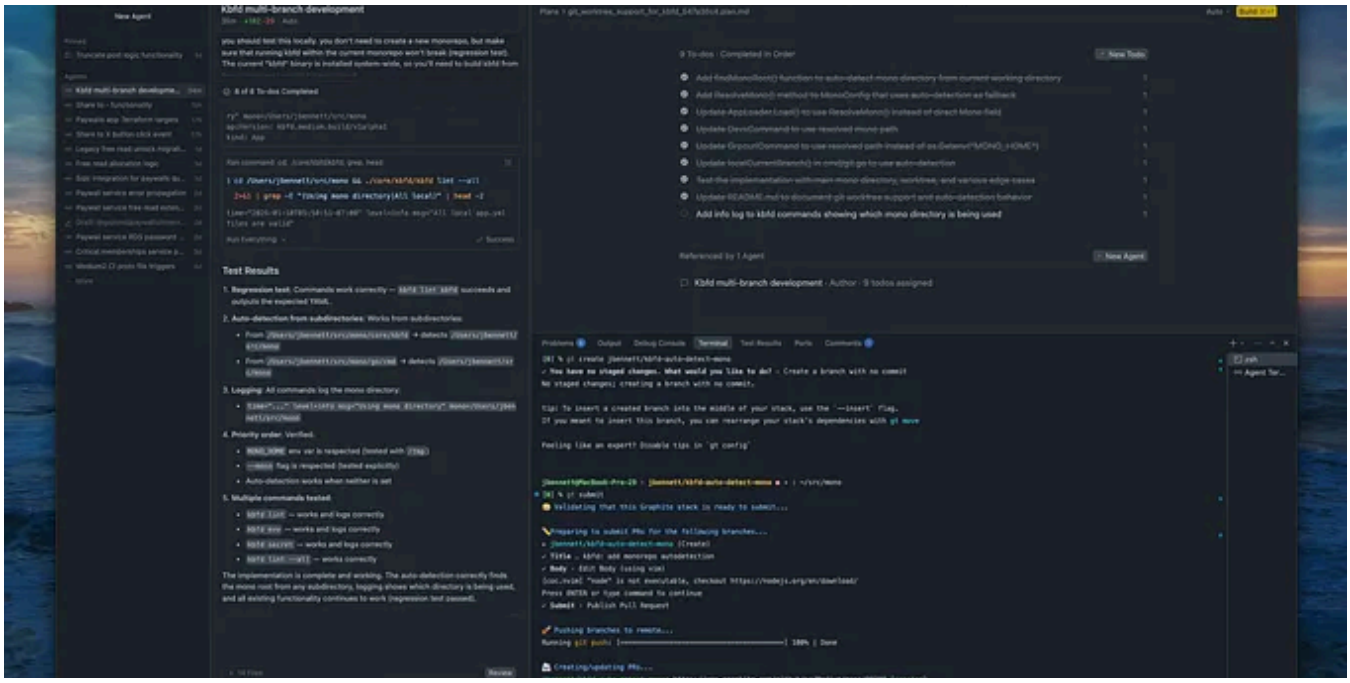


Heeki Park

Using spec-driven development with Claude Code

I no longer write code by hand. I wouldn't call myself a proper software development engineer by trade, nor have I deployed large-scale...

Mar 1 506 13



Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software Engineer

Tools I use that are (usually) cheaper than Netflix

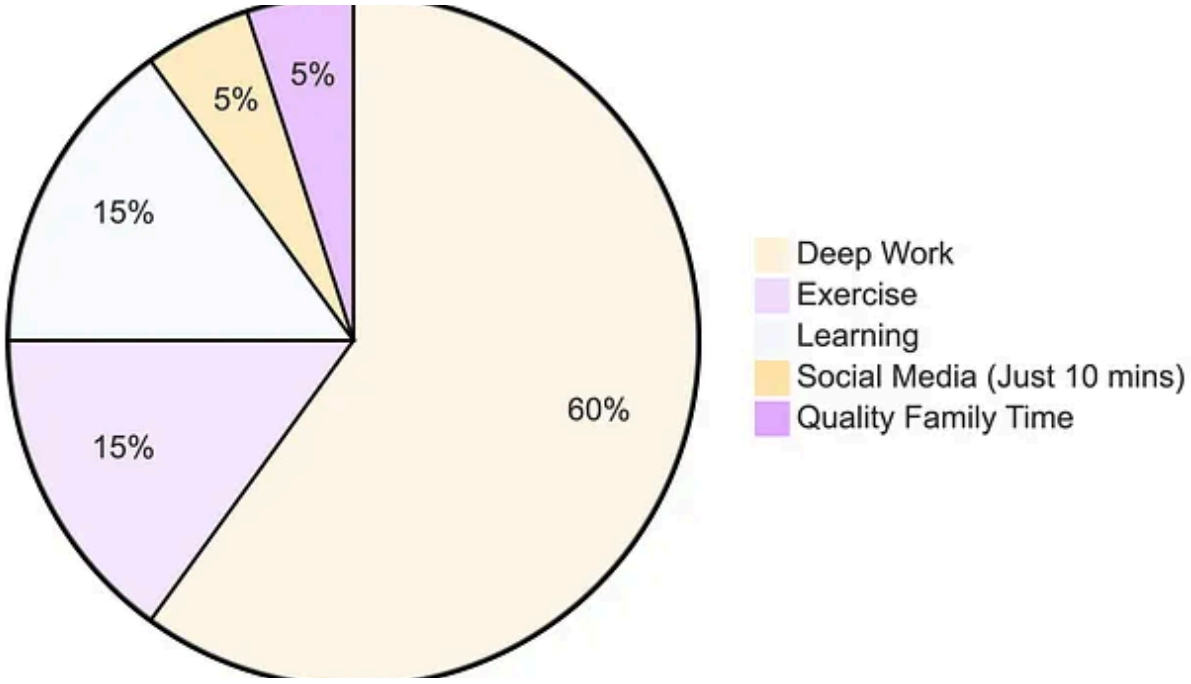
Jan 19 4.1K 99

In Stackademic by HabibWahid

Junior Devs Use try-catch Everywhere. Senior Devs Use These 4 Exception Handling Patterns

Try-catch on every method? That's not safe code—that's a ticking time bomb. Here's what senior devs do instead.

Feb 1 1.7K 34



In Level Up Coding by Teja Kusireddy

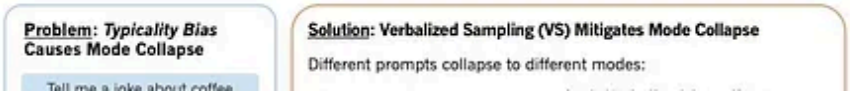
I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

Dec 28, 2025 11.3K 407

Abstract

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce ***Verbalized Sampling (VS)***, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., "Generate 5 jokes about coffee and their corresponding probabilities"). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.



In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required...

Oct 20, 2025 25K 674